Sky Custom Reports — Handover Document

> Draft v1. Prepared on the departure of Sergei Privalov (Backend Engineer, project owner). Original written in Russian (`handover-ru.md`); this is the English version.

---

1. Roles and Responsibilities

Role: Backend Engineer, sole owner and primary developer of the `sky-custom-reports` service since the project was created (first commit — August 23, 2025).

Responsibilities:

- Full development lifecycle of the service: architecture design, API and workers implementation, database schema, CI/CD, deployments.
- Ticket estimation and work planning for the project (Linear, SKY-* prefix).
- Review and ownership of all features, including contributions from other team members.
- Staging/production deployments (manual, see §8).
- Minimal stakeholder/customer communication — occasional only.

People and contacts:

- **Victor Karpov** — project supervisor at the start; proposed the initial architectural decisions (Node.js, Redis, separate workers). Good contact for "why was it done this way" questions about the early stage.
- **Stas Kulesh** — second-largest contributor (~40 commits), worked mostly via an AI-assisted approach (Devin): ad-hoc extractions, AI-edit for sections/reports, chronology sections.
- Vlad, Viktor, Gleb — occasional commits.
- Devin AI — ~14 bot commits.

The project has no formal team — the successor becomes the sole owner, as the author of this document was.

---

2. Application Overview

2.1 Purpose

A microservice for generating template-based reports using LLMs (Google Gemini — primary provider). The service's only client is the external `usesky-api` (the main backend of the Sky product); the service has no authentication of its own — trust is at the network level.

Typical flow: a user in the frontend application (**app**) triggers report generation for a "case" → `app` → `usesky-api` → `sky-custom-reports` API → the service fetches the case document list (doclist), fans the work out across template sections, each section is generated by an LLM pipeline that extracts facts from documents and attaches source citations (RAG sources) → the finished report is read back via the v3 API (app ← usesky-api ← sky-custom-reports).

2.2 Architecture

```
app → usesky-api → API (NestJS :3003) → BullMQ (Redis) → Workers (Genkit/LLM)
                                                ↘ PostgreSQL (Prisma), S3 (doclist,
prompts), sky-rag (pgvector)
```

- **api/** — NestJS 11 + Prisma 6, BullMQ producer, Bull Board (`/admin/queues`), Swagger (`/api`).
- **workers/** — Node 22 + tsx, BullMQ consumers + Genkit (Gemini primary, Claude fallback, Vertex AI for the NA instance).
- **packages/database**, **packages/queues** — shared PrismaClient and queue contract (names, Zod job schemas, TTLs, NA namespace `na_`).
- **prisma/** — single `schema.prisma` + ~38 migrations, schema shared by api and workers.

2.3 Queues and workers

Queue	Worker	Purpose
`reports-queue`	`reportsGenerationWorker`	Report fan-out: prepares doclist and section snapshots in Redis (TTL 4h), enqueues a job per section
`sections-queue`	`reportSectionGenerationWorker`	Report section generation; also ad-hoc and preview jobs
`section-page-chunks-queue`	`sectionChunkWorker`	Page chunks for large snapshot sections (>400 pages)
`template-generation-queue`	`templateGenerationWorker`	Template generation from sample PDF; `report-evaluation` job
—	`templateOrphanReconciler`	Marks "stuck" processing templates via `heartbeatAt`

Entry point: `workers/src/index.ts` (S3 check, Vertex auth in NA mode, `registerPrompts()`, OpenRouter pricing warm-up, starts 5 workers).

2.4 Key features and their owners

Feature	Author	Notes
Multi-stage section pipeline (chunking → relevance → extract facts → merge → prose + citations)	Sergei	Second iteration of the approach; the first worked from section summaries

Feature	Author	Notes
Template generation from sample PDF	Sergei	`generateTemplateFlow`
Cross-section fact dispatch/dedup	Sergei	flag `CROSS_SECTION_DEDUP_ENABLED` — should be enabled in prod now
Eval infrastructure (chronology, report-fidelity, report-gold)	Sergei	see §6
Ad-hoc extractions	Stas	`POST /v1/extractions/ad-hoc`, synchronous via Redis result-store
AI-edit for sections/reports	Stas	`POST /v3/cases/:caseId/reports/:id/ai-edit` and related endpoints
Chronology sections (`contextType=chronology`)	Stas	`generateChronologySectionFlow`
RAG sources (document-page citations)	Sergei	see §3.1

---

3. Approach and Implementation Details

3.1 Section generation pipeline (contextType = snapshot)

Current approach: a section is generated **from the actual page content of documents**, not from summaries (the early version worked from summaries — that proved insufficient for source citations).

Stages (`workers/src/genkit/flows/report-section/`):

1. **Chunking** — splitting the doclist into page chunks; above 400 pages — fan-out to `section-page-chunks-queue`.
2. **detectRelevanceStep** — the LLM determines page relevance in batches (`relevanceCache`, fail-open: on a batch error the pages are kept).
3. **fetchPagesStep** — loads page markdown directly from S3 (the service reads S3 directly, bypassing intermediate layers) and/or via **sky-rag** (`POST /v1/tenants/{org}/cases/{case}/context`, pgvector). RAG is effectively used as a page-selection filter at large volumes — a compromise, not a full RAG context.
4. **extractChunkFactsFlow** — atomic fact extraction from chunks.
5. **mergeFactsTreeStep** — hierarchical fact merge/dedup (tournament/condensedFacts).
6. **finishSnapshotSectionFlow** — prose generation, `buildMdastStep` → `contentMdast`, `buildSpansStep`, `mergeSourcesStep` (citation), `resolveSectionLinksStep`.

Separate flows: `generateDocListSectionFlow`, `generateChronologySectionFlow`, `humanInput` (a section filled in by the report author).

The key unsolved problem: exhaustive fact extraction, readable presentation, and preserving citations at the same time — the hardest part of the system; quality is constantly being balanced.

Section fan-out was chosen because sections can be generated in parallel (a template may have ~50 sections) — this substantially reduces report generation time. Not implemented: a final "whole-report pass" for cross-section text coherence.

3.2 LLM layer: Genkit + prompts

Genkit (Google): chosen as a single interface to different LLMs, convenient manual tracing, structured input/output (Zod schemas). Wrapper: `workers/src/genkit/prompts/prompt.ts` — each prompt has a main model + fallback model; config comes from `config.json` (see below). Known issue: Genkit appears to accumulate registered schemas in memory — workers show slow memory growth and need periodic restarts. Worth verifying/confirming, and possibly revisiting the use of Genkit (including newer agentic approaches that have emerged since).

Models: primary — Gemini (`gemini-3.5-flash(-lite)`, `gemini-3.7/3.8-flash`, `gemini-3.1-pro-preview`), fallback — Claude (`haiku-4-5` / `sonnet-4-6` / `opus-5`). Pricing — `infra/modelPricingTable.ts` table plus warm-up of current prices from OpenRouter into Redis at startup. The NA instance uses the `vertex-ca` config via Vertex AI.

3.3 Prompts: sky-prompts repo → S3

Prompts are LiquidJS templates living in the `sky-prompts` repository (`prompts/custom-reports/`). On merge to `develop`, CI publishes them to S3 (`S3_SETTINGS_BUCKET` + `S3_PROMPTS_PREFIX=custom-reports`). Alongside the templates sits a single **`config.json`** defining, per prompt, the main/fallback/vertex-ca model, generation parameters, attempt count, and timeout.

Important: `config.json` and templates are **loaded once at worker startup and cached in memory** (`s3ConfigCache` in `workers/src/infra/promptLoader.ts`). The original intent was hot-updating configuration via S3 — in practice this does not happen, since applying changes requires a restart. **Recommendation:** move prompt version/config selection into code or env variables — the current scheme creates a false expectation of hot updates.

Local development without S3: `USE_LOCAL_PROMPTS=1` + `LOCAL_PROMPTS_PATH=<path to a sky-prompts clone>/prompts/custom-reports`.

3.4 Retry layers — a major source of complexity

The system has **several nested retry levels**; their imbalance has already caused incidents:

1. **In-process LLM-call retries** — in `prompt.ts`: `mainAttempts` on the main model, then `fallbackAttempts` on the fallback (exponential backoff 2–30s). Per-prompt attempts/timeoutMs come from `config.json`.
2. **BullMQ job retries** — `packages/queues/src/job-options.ts`: ~3–4 attempts per job (report/section/chunk/template). Parent and child jobs retry independently: if a child exhausts attempts without `failParentOnFailure`, the parent can hang or fail separately.

3. **S3 retries** — `infra/s3.ts`: exponential backoff on object reads.
4. **Idempotency** — `Report.idempotencyKey` (unique) + a partial unique index `(reportId, idempotencyKey)` on ReportSection, created **manually in a migration** (not via Prisma DSL — a fragile spot).

Rule: `AllAttemptsExhaustedError` is treated as a permanent error so the queue does not start another in-process retry cycle. When changing any level, check the product of attempt counts — multiplied retries caused the historical cost incident.

3.5 Redis as a data bus, not just a queue

Redis holds: BullMQ queues, prepared doclist and section snapshots (`report:prepared-doclist:`, `report:section-snapshot:`, TTL 4h), page-chunk job results (`report:section-chunk-results:`), ad-hoc/preview result-store, relevance cache, OpenRouter price cache. The 4h TTL is a failure point: if a report sits in the queue longer than the TTL, snapshots expire.

---

4. Infrastructure, Deployment, and External Dependencies

4.1 Deployment

- **CI:** GitHub Actions on self-hosted `[sky-ops]` runners, Node 22. `pr.yml` / `_tests.yml`: lint + typecheck + format → unit tests (Jest for api, Vitest for workers) → integration job (brings up the full docker-compose, api e2e + `npm run test:integration`).
- **Deploy:** `deploy-development/staging/production.yml` → docker build → push to **ECR** → `aws ecs update-service --force-new-deployment` on **ECS** (prod: `sky-prod-cluster`, services `sky-prod-custom-reports-api/-worker`, ca-central-1). The NA service `<svc>-na` deploys the same way in parallel.
- **Prod deploys are manual**: workflow_dispatch with `DEPLOY` confirmation, tag `deployed/production`.
- **Schedule:** deploy **on Tuesdays**: staging first, then prod; the service deploy is synchronized with the prompt release (merge in sky-prompts → auto-publish to S3).
- Prisma migrations run in `docker-entrypoint.sh` at api container startup.

4.2 NA instance

`IS_NA_INSTANCE=1` — a separate deployment in Canada (data residency requirements). All queues and Redis keys get the `na_` prefix; LLM goes only through Vertex AI (`VERTEX_AI_BASE_URL`, WIF authentication, `infra/vertexAuth.ts`). **API and workers must have the same flag value.** The instance is live in prod and deploys in parallel with the main one.

4.3 External dependencies

- `usesky-api` (core API) — the only client; `api/src/core-api/core-api.client.ts`.

- `sky-rag` — vector page search (`SKY_RAG_URL`, requires VPN); used as a page-selection filter at high page counts.
- S3 buckets: doclist/document pages (direct access), settings bucket with prompts.
- LLM: Google Gemini (primary), Anthropic (fallback), Vertex AI (NA).
- Monitoring: Bull Board (`/admin/queues`), JSON logs → CloudWatch → Grafana (dashboard see Appendix; some panels were built for the old pipeline — see §7).

4.4 Metrics, logs, and investigating failed generations

All metrics are structured JSON logs (`workers/src/infra/metrics.ts`, `component: "metrics"`, `metric` field) that flow into CloudWatch and are visualized in Grafana:

Dashboard: https://grafana.sky-tech.ai:8843/dashboards/f/ff3a7eddosjy8f/?orgId=1 (Environment: prod-cloudwatch). Panel sections:

- **Evaluation Quality** — eval run results (`metric: evaluation_result`: score/passed by evaluatorName).
- **Performance** — job/generation durations.
- **Report Generation Insights** — key product metrics: Avg Generation Time, Avg Pages / Case, Avg Sections / Report, Avg Cost / Report and their trends over time. Source — `report_completion` and `job_completion` (`duration_ms`, `usdCost`, `docListPageCount`, `totalSections` fields).

Metric types (values of the `metric` field in logs):

metric	What it records	Key fields
`job_completion`	completion of each job (report/report_section/ section_chunk/template/ ad_hoc/preview/ media_preview)	`status`, `duration_ms`, `reportId`, `reportSectionId`, `usdCost`, `docListPageCount`, `inputTokens`/ `outputTokens`, `factsBeforeDispatch`/ `factsAfterDispatch`
`report_completion`	report status recompute	`reportId`, `status`, `totalSections`, `duration_ms`, `usdCost`, `templateName`, `organizationId`
`llm_request`	each LLM call	`provider`, `model`, tokens, `usdCost`, `attempt`, `isRetry`, `contextId`
`llm_token_usage`	per-prompt tokens at flow completion	`flowName`, `promptName`, `modelName`, tokens
`llm_fallback`	main→fallback model switch	`promptName`, `mainModel`, `fallbackModel`, `mainAttemptsMade`
`retry`	each retry (llm_request/ job_processing/ redis_connection)	`operation`, `attempt`, `maxAttempts`, `delay_ms`, `reason`
`error`	classified error	`errorType` (LLMError/NetworkError/ ValidationError/DatabaseError/ UnknownError), `reportSectionId`, `jobId`
`job_processing`		

metric	What it records	Key fields
	start/finish of a specific section job	`jobId`, `reportSectionId`, `status`, `attempt`
`cost_aggregation`	periodic cost rollup	`provider`, `model`, `totalUsdCost`, `period_minutes`
`fact_attribution`	share of facts from pages vs summary	`totalFacts`, `pagePercentage`, `summaryPercentage`, `sectionTitle`
`source_coverage`	citation coverage of facts in the response	`coverageRate`, `coveredFacts`/`totalFacts`
`cross_section_dedup`	cross-section dedup metrics	`reportId`, `runId`
`orphan_reconciled`	row transitioned to failed by the reconciler	`entityType`, `entityId`, `jobState`
`evaluation_result`	eval run result	`evaluatorName`, `score`, `passed`

How to trace a failed generation:

1. **Find the reportId** — from the report URL in the app, from Bull Board (`/admin/queues`), or from the API response.
2. Filter CloudWatch Logs by `reportId` — the whole chain is visible: fan-out → section jobs → `report_completion` with status (`processing`/`completed`/`failed`) and total cost.
3. Find failed sections: `metric = job_completion AND status = failed` (or `job_processing` with `status=failed`) → `reportSectionId` → all `error`/`retry`/`llm_request` entries for it.
4. Pattern-based diagnosis:
5. many `llm_fallback` for one `promptName` → main-model degradation (Gemini);
6. a spike of `retry` with `operation=llm_request` → LLM incident; check the product of attempt counts (see §3.4);
7. `orphan_reconciled` → a job died without recording a result (worker restart/crash);
8. `report_completion status=processing` longer than STALE_THRESHOLD (15 min) → stuck report, `checkAndUpdateStaleReport`.
9. Cost: `llm_request` by `contextId`/job and `usdCost` on `job_completion`/`report_completion` — where the budget burned; globally — `cost_aggregation` and `/admin/analytics/token-usage`.

4.5 Tests

- API: Jest unit (`*.spec.ts`) + e2e `api/test/*.e2e-spec.ts` (`npm run api:test`, `test:e2e`).
- Workers: Vitest (`npm run workers:test`); `test:llm` — runs against real LLMs (`__realtests__/*.realllm.test.ts`).
- Integration: `tests/summary-generation-test.integration.ts` (`npm run test:integration`, needs the full stack + LLM keys).
- Eval: see §6.

---

5. Problems and Incidents (history)

1. **Retry multiplication → expensive incidents.** During LLM failures/degradation, unbalanced retries at the LLM-call level × job retries produced a request avalanche; on large cases generation never finished while costs spiked. Resolution: permanent-error classification (`AllAttemptsExhaustedError`), attempt caps, heartbeat mechanisms.
2. **"Stuck" processing rows.** No reliable way to tell a job is truly dead. Partially solved: `templateOrphanReconciler` + `heartbeatAt` for templates, lazy `checkAndUpdateStaleReport` (STALE_THRESHOLD 15 min) for reports, report heartbeat in the section worker (SKY-3400). **Stage 6 — heartbeat/reconciler for section jobs — not implemented** (`docs/stuck-processing-remediation-plan.md`).
3. **Summary-first approach proved insufficient** — no way to cite pages. Migrated to the page-content pipeline (current, second iteration). Scaling validated up to roughly ~1000 pages; 5–10k pages are not validated — a known scaling limit.
4. **504 timeouts on the reports endpoint** — branch `SKY-1205/fix-...-timeout-504`.
5. **Citation corruption** — fixes under SKY-3944 (cindy citation), partly related to mergeSourcesStep/buildSpansStep.
6. **Genkit memory growth** — see §3.2; needs confirmation and periodic worker restarts.
7. **Objective quality measurement** — absent until recently; hence the eval infrastructure (§6).

---

6. Eval Infrastructure (in progress)

Goal — objectively measure generation quality. Three tracks:

- **chronology eval** — `workers/src/eval/chronology/` + `workers/scripts/eval/chronology/` (`eval:chronology`, SKY-2948): matcher/scorer/grounding/reliability.
- **report-fidelity eval** — server-side pipeline: `workers/src/report-eval/` + v3 API `report-evaluations` + `report-evaluation` job + CLI `scripts/eval/report-fidelity/runner.ts` (polls the API). Judge is an LLM (`reportFidelityJudgePrompt`). Spec `specs/005-full-report-eval/spec.md` is marked implemented but describes an outdated in-process design — the docs diverge from the code.
- **report-gold eval** — current uncommitted work on branch `devin/sky-3944-report-gold` (SKY-3944): gold report set (cindy, john-doe, stolerman-karp), `workers/src/eval/report/` (judge, scorer, snapshot, types) + `workers/scripts/eval/report-gold/` (run/evaluate/pipeline/judgeClient/exportSnapshot). Judge model — `gemini-3.8-flash`, fact batching, judge-response caching, cost tracking.

Limitation: fully autonomous eval is blocked on access — a run needs AWS S3 and the dev DB; the AI assistant (Devin) does not have these, so some steps are manual. Resolve access — either grant it or keep the semi-manual process.

---

7. Pending Work and Recommendations

Active/WIP

1. **SKY-3944, branch `devin/sky-3944-report-gold`** — uncommitted report-gold eval files (see `git status`). Commit/push before handover; bring to a reproducible run.
2. **Stage 6 stuck-processing** (`docs/stuck-processing-remediation-plan.md`) — heartbeat/orphan-reconciler for section jobs, following the template reconciler pattern.
3. **`CROSS_SECTION_DEDUP_ENABLED`** — should be enabled in prod; **add the flag to the infra configuration** (currently default-off in code).
4. **Frontend round-trip** for template-section identity — backend ready (`docs/preserve-templatesection-identity-plan.md`), waiting on the frontend editor.
5. **Dead script:** `workers/package.json → eval:report-fidelity` points to a nonexistent `workers/scripts/eval/report-fidelity/runner.ts` (the live runner is in the root `scripts/`). Remove or fix.
6. **README is partially outdated:** describes the old queue scheme, missing chronology/humanInput ContextTypes, current APIs are v3/v4 (v1/v2 deprecated but live).
7. **Grafana dashboard** for the current pipeline — to be finished.
8. **Possible duplicate:** `constants.ts` and `constants/reports.constants.ts` in `api/src/reports`.

Strategic recommendations (author's opinion)

- **Consider migrating queues BullMQ → SQS.** Redis was chosen at the start (Victor's suggestion); SQS is already used by other Sky services; parent/child jobs on BullMQ are not the best model — SQS would simplify reliability and the stuck-job problem.
- **Cache/warm shared LLM context** during report preparation (avoid resending identical context to the LLM) — cost savings.
- **Revisit Genkit** — memory growth, plus newer agentic approaches now exist.
- **Remove "hot" prompt configuration from S3** in favor of code/env (§3.3) — the current scheme implies hot-update that doesn't actually happen.
- **Final report pass** (stitching/coherence across sections after parallel generation) — not implemented.
- **5–10k page scaling** — validate the pipeline and cost.
- **Enable and roll out cross-section dedup in infra.**
- **Facts as a persistent data layer (long-term direction).** Today facts are re-extracted on every generation and live only inside a single run. Recommended: extract facts at pipeline stages and **persist them in the DB with structure and metadata** (source, page, date, type — medical/chronology/insurance event, etc.), deduplicate across runs and sections, then **retrieve facts per user queries** (semantic/graph search, topic clustering — medical, chronology, etc.) and assemble a coherent report from them. This maps naturally onto **agentic approaches**: an agent picks relevant fact clusters and composes the report,

instead of a rigid linear pipeline. It addresses the core quality problem (§3.1) and makes repeat generations cheaper.

Routine operations for the successor

- Tuesday deploys: staging → prod (workflow_dispatch + `DEPLOY`), in sync with prompt releases from sky-prompts.
- Investigating stuck/failed jobs — Bull Board `/admin/queues`.
- Checking stuck `processing` rows — DB + logs, `checkAndUpdateStaleReport`.
- Updating prompts — PR in `sky-prompts` → merge to develop → auto-publish to S3 → worker restart to apply.
- DB migrations — `prisma/migrations`, auto-run at api startup.
- Cost monitoring — `/admin/analytics/token-usage`, `usdCost` fields on ReportSection.

---

8. Access

Resource	What's needed
AWS (ECR/ECS/S3/CloudWatch), accounts dev `418272754107`, prod `717279713537`	Cluster access, doclist/settings/prompts buckets
GitHub: sky-custom-reports, sky-prompts, infra, usesky-api	Write + workflow_dispatch
Google/Gemini API key, Anthropic API key	Local development and test:llm
GCP/Vertex (WIF)	NA instance
Redis/Postgres dev	Locally via docker-compose; dev environment — via DevOps
VPN	sky-rag access
Linear SKY-*	Project board

Appendix. Key paths in the repository

- API controllers: `api/src/{templates,reports,report-evaluations,previews,extractions,analytics,health}/` — current versions are v3/v4.
- Section pipeline: `workers/src/genkit/flows/report-section/`.
- Prompt wrapper: `workers/src/genkit/prompts/prompt.ts`; loading: `workers/src/infra/promptLoader.ts`.
- Queue contract: `packages/queues/src/` (job-options.ts — retries).
- Eval: `workers/src/eval/{chronology,report}/`, `workers/scripts/eval/`, `scripts/eval/report-fidelity/`.
- Design docs: `docs/stuck-processing-remediation-plan.md`, `docs/preserve-templatesection-identity-plan.md`, `workers/TEMPLATE_GENERATION.md`, `VERIFICATION.md`.
- CI/CD: `.github/workflows/`.

Appendix. Links

- Grafana dashboard (prod): https://grafana.sky-tech.ai:8843/dashboards/f/ff3a7eddosjy8f/?orgId=1
- Bull Board (queues): `https://<api-host>/admin/queues`
- Swagger: `https://<api-host>/api`
- Token analytics: `GET /admin/analytics/token-usage`
- Prompts repository: `sky-prompts` → `prompts/custom-reports/` (+ `config.json`)